MEMORIA PRÁCTICA FINAL

MÁQUINA EXPENDEDORA DE BEBIDAS

Patricia Cárdenas Fernández
3º Computadores A

ÍNDICE

1. INTRODUCCIÓN DEL PROYECTO 2

2. MATERIALES Y HERRAMIENTAS UTILIZADOS 2

3. FUNCIONAMIENTO Y MONTAJE 3

4. ANOTACIONES DEL CÓDIGO 4

4.1. RESUMEN FUNCIONES DEFINIDAS 4

4.1.1. BCD_InicializarCEx 4
4.1.2. BCD_apagarDisplay 5
4.1.3. BCD_enviarDigito 5
4.1.4. BCD_enviarLetra 6
4.1.5. Matrix_leerTecla 6
4.1.6. Matrix_inicializarFilas 7
4.1.7. char_to_int 7

4.2. NUESTRO BUCLE WHILE 7

ÍNDICE DE FICGURAS

Ilustración 1 3

Ilustración 2 4

1. INTRODUCCIÓN DEL PROYECTO

La elección del proyecto final de la asignatura ha sido una Máquina Expendedora, concretamente de bebidas, aunque también se puede adaptar a cualquier otro tipo de máquina expendedora, como de tickets, de snacks, de souvenirs... Este proyecto simula el funcionamiento de una Máquina Expendedora de bebidas básica, tanto la marcación de teclas para elegir el producto, como la muestra de la bebida y su correspondiente precio por pantalla, así como la introducción del precio y su posterior cálculo para la devuelta. En el apartado *FUNCIONAMIENTO Y MONTAJE* se explica más detalladamente el funcionamiento del proyecto, pero veamos un resumen de sus características principales:

- Funciona de manera reiterada, es decir, se puede volver a pedir un producto una vez ha acabado una operación con éxito.
- Sólo contiene tres bebidas: Zumo, té y cola.
- Los productos se muestran en pantalla uno a una al pulsar la tecla A.
- Se selecciona un producto pulsando la tecla B, mostrando su correspondiente precio.
- La introducción de monedas se simula con la pulsación de las teclas del teclado.
- Se muestra la devuelta, en caso de introducir un precio mayor al del producto, o el dinero restante, en caso contrario. Si muestra 00.00, significa que se ha introducido el dinero exacto.

2. MATERIALES Y HERRAMIENTAS UTILIZADOS

Para mostrar la simulación del proyecto de ha utilizado una Placa de Periféricos adaptada a la asignatura. Esta contiene elementos típicos de testeo, tales como un teclado matricial, pulsadores, switches, leds, displays BCD 7-Segmentos y un display LCD. Todos estos componentes pueden ser controlados a través de un conector **DIN41612** de 64 pines.

En el proyecto se han usado los siguientes elementos de la placa:

- Displays 7-Segmentos.
- Led de signo.
- Teclado Matricial.

Para tener control sobre los distintos componentes de la Placa de Testeo necesitamos programar en C el comportamiento que éstos deben tener. Para ello hemos utilizado un microcontrolador **STM32 nucleo**, el cual se enlaza al conector **DIN41612** de la Placa de Testeo a través de una serie de cables.

Además, necesitamos configurar el microcontrolador para que sea capaz de suministrar las señales a los componentes de la Placa de Testeo. El programa **STM32CubeMX** nos facilita la configuración e inicialización de los pines que queremos usar. Una vez configurados los pines del microcontrolador, pasamos a programar el funcionamiento en un entorno de programación, como es **eclipse**.

3. FUNCIONAMIENTO Y MONTAJE

En el apartado *INTRODUCCIÓN DEL PROYECTO* se han resumido las características principales que cumple el proyecto. A continuación, se explica más detalladamente el funcionamiento completo de la Máquina Expendedora de Bebidas:

1. Se muestra un mensaje de bienvenida en los displays 7-Segmentos durante 2 segundos.
2. Aparece el primer producto (zumo) en los displays 7-Segementos.
3. A partir de ahora se puede pulsar repetidamente la tecla *A* para ir mostrando los siguientes productos uno a uno.
4. Para elegir uno de los productos, se pulsa la tecla *B*.
5. Aparece el precio en los displays 7-Segementos del producto seleccionado.
6. Tras 1 segundo de espera, se puede comenzar a introducir el precio a través del teclado matricial. Es necesario introducir 4 números exactamente, 2 para los euros y otros 2 para los céntimos.
7. Se calcula la operación y se muestra en los displays 7-Segmentos la devuelta si el dinero introducido supera el precio del producto, o el dinero que aún queda por introducir si el dinero introducido no supera o iguala el precio del producto. En el caso de la devuelta, esta se identifica diferencia por el led de signo, que se mantiene activado.
8. Una vez “pagado” el producto, se vuelve a mostrar el mensaje de bienvenida transcurridos 5 segundos.

A continuación, se muestra una imagen final del proyecto en funcionamiento:

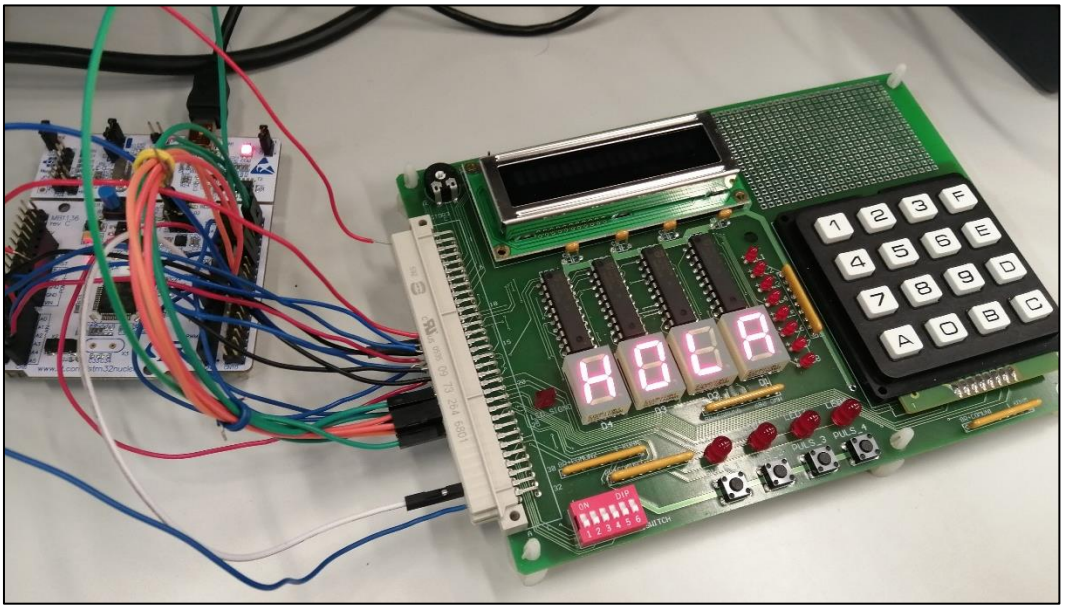


Ilustración 1 Montaje final del proyecto en funcionamiento

4. ANOTACIONES DEL CÓDIGO

4.1. FUNCIONES DEFINIDAS

En la siguiente taba se muestran las funciones definidas y su correspondiente comportamiento. **NOTA**: a partir de ahora, al hablar de FILAS nos referimos a COLUMNAS, y viceversa, debido a que, en la documentación del teclado, parece ser que no se asemeja al diseño real. Más adelante se muestra un esquema del teclado.

Prototipo de función	Función
`static void BCD_inicializarCEx (void)`	Desactiva el pin de control de cada uno de los 4 displays, de manera que no muestren el contenido guardado en ese momento en sus correspondientes registros.
`static void BCD_apagarDisplay (void)`	Apaga todos los displays.
`static void BCD_enviarDigito (int, int, int, int)`	Muestra un número en un display determinado. Dependiendo del caso, puede encender el led de signo o el punto.
`static void BCD_enviarLetra (char, int)`	Muestra una letra en un display determiando.
`static char Matrix_leerTecla(void)`	Realiza un sondeo por todas y cada una de las filas y columnas (excepto la última fila) y devuelve el carácter correspondiente a una tecla cuando detecta su pulsación.
`static void Matrix_inicializarFilas(void)`	Envía un 0 al pin de cada fila para la siguiente iteración
`static int char_to_int(char)`	Convierte un carácter número en un entero.

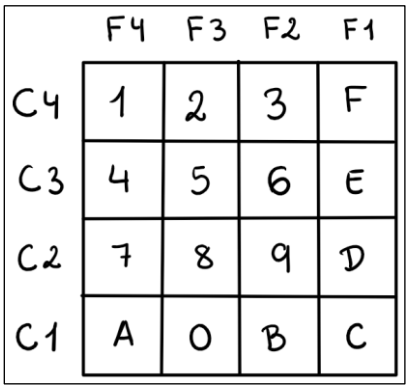


Ilustración 2 Esquema teclado matricial

4.1.1. BCD_inicializarCEx

```
static void BCD_inicializarCEx (void)
{
    HAL_GPIO_WritePin(CE1_GPIO_Port, CE1_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(CE2_GPIO_Port, CE2_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(CE3_GPIO_Port, CE3_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(CE4_GPIO_Port, CE4_Pin, GPIO_PIN_SET);
}
```

Esta función se encarga de inicializar los pines de control de los cuatro dislpays 7-Segmentos. Curiosamente, para desactivarlos, le enviamos un 1 lógico, no un 0.

4.1.2. BCD_apagarDisplay

El comportamiento de esta función es apagar los cuatro displays a la vez. Para ello enviamos un 0 lógico a cada uno de los siete segmentos de cada display y, seguidamente, activamos los cuatro pines de control para que reciban la configuración. A continuación, se vuelven a desactivar hasta un posterior cambio en la configuración.

```
static void BCD_apagarDisplay (void)
{
    HAL_GPIO_WritePin(DISP0_GPIO_Port, DISP0_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(DISP1_GPIO_Port, DISP1_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(DISP2_GPIO_Port, DISP2_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(DISP3_GPIO_Port, DISP3_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(DISP4_GPIO_Port, DISP4_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(DISP5_GPIO_Port, DISP5_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(DISP6_GPIO_Port, DISP6_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(DISP7_GPIO_Port, DISP7_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(SIGNO_GPIO_Port, SIGNO_Pin, GPIO_PIN_RESET);

    /* Habilitamos todos los CEx */
    HAL_GPIO_WritePin(CE1_GPIO_Port, CE1_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(CE2_GPIO_Port, CE2_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(CE3_GPIO_Port, CE3_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(CE4_GPIO_Port, CE4_Pin, GPIO_PIN_RESET);

    /* Volvemos a deshabilitar los CEx */
    HAL_GPIO_WritePin(CE1_GPIO_Port, CE1_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(CE2_GPIO_Port, CE2_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(CE3_GPIO_Port, CE3_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(CE4_GPIO_Port, CE4_Pin, GPIO_PIN_SET);

}
```

4.1.3. BCD_enviarDigito

```
static void BCD_enviarDigito (int character, int display, int punto, int signo)
{
    switch (character) {
    case 0:
        HAL_GPIO_WritePin(DISP0_GPIO_Port, DISP0_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP1_GPIO_Port, DISP1_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP2_GPIO_Port, DISP2_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP3_GPIO_Port, DISP3_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP4_GPIO_Port, DISP4_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP5_GPIO_Port, DISP5_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP6_GPIO_Port, DISP6_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP7_GPIO_Port, DISP7_Pin, GPIO_PIN_RESET);
        break;
    case 1:
        HAL_GPIO_WritePin(DISP0_GPIO_Port, DISP0_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP1_GPIO_Port, DISP1_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP2_GPIO_Port, DISP2_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP3_GPIO_Port, DISP3_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP4_GPIO_Port, DISP4_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP5_GPIO_Port, DISP5_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP6_GPIO_Port, DISP6_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP7_GPIO_Port, DISP7_Pin, GPIO_PIN_RESET);
        break;
```

Esta es una de las funciones principales del código, ya que es la encargada de mostrar un dígito en el display correspondiente. En primer lugar, debemos obtener la configuración correspondiente a *character* que, en este caso, será un número del 0 al 9. En la imagen sólo se muestra el caso del 0 y el 1.

```
if (signo) {
    HAL_GPIO_WritePin(SIGNO_GPIO_Port, SIGNO_Pin, GPIO_PIN_SET);
}

if (display == 1) {
    HAL_GPIO_WritePin(CE1_GPIO_Port, CE1_Pin, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_RESET);
    HAL_Delay(10);
    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_SET);
    HAL_Delay(10);

    HAL_GPIO_WritePin(CE1_GPIO_Port, CE1_Pin, GPIO_PIN_SET);

} else if (display == 2) {
    HAL_GPIO_WritePin(CE2_GPIO_Port, CE2_Pin, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_RESET);
    HAL_Delay(10);
    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_SET);
    HAL_Delay(10);

    HAL_GPIO_WritePin(CE2_GPIO_Port, CE2_Pin, GPIO_PIN_SET);
```

En la primera condición lo que se comprueba es si queremos encender el led de signo (indicado por el parámetro *signo*). Una vez tenemos la configuración, la enviamos al display que indica el parámetro *display*. Para mostrar el dígito, basta con habilitar el pin de control del display y deshabilitarlo justo después, siempre en un ciclo de reloj.

4.1.4. BCD_enviarLetra

Esta función tiene exactamente el mismo comportamiento que la función *BCD_enviarDigito*, sólo que, en lugar de números, son letras lo que debemos comprobar a partir del parámetro *character*.

```
static void BCD_enviarLetra (char character, int display)
{
    switch (character) {
    case '0':
        HAL_GPIO_WritePin(DISP0_GPIO_Port, DISP0_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP1_GPIO_Port, DISP1_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP2_GPIO_Port, DISP2_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP3_GPIO_Port, DISP3_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP4_GPIO_Port, DISP4_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP5_GPIO_Port, DISP5_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP6_GPIO_Port, DISP6_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP7_GPIO_Port, DISP7_Pin, GPIO_PIN_RESET);
        break;
    case 'Z':
        HAL_GPIO_WritePin(DISP0_GPIO_Port, DISP0_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP1_GPIO_Port, DISP1_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP2_GPIO_Port, DISP2_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP3_GPIO_Port, DISP3_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP4_GPIO_Port, DISP4_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP5_GPIO_Port, DISP5_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(DISP6_GPIO_Port, DISP6_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(DISP7_GPIO_Port, DISP7_Pin, GPIO_PIN_RESET);
        break;
```

Al igual que en la función anterior, una vez obtenida la configuración del carácter, la enviamos al display correspondiente al parámetro *display*, siempre haciéndolo en un ciclo de reloj.

```
if (display == 1) {
    HAL_GPIO_WritePin(CE1_GPIO_Port, CE1_Pin, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_RESET);
    HAL_Delay(10);
    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_SET);
    HAL_Delay(10);

    HAL_GPIO_WritePin(CE1_GPIO_Port, CE1_Pin, GPIO_PIN_SET);

} else if (display == 2) {
    HAL_GPIO_WritePin(CE2_GPIO_Port, CE2_Pin, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_RESET);
    HAL_Delay(10);
    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_SET);
    HAL_Delay(10);

    HAL_GPIO_WritePin(CE2_GPIO_Port, CE2_Pin, GPIO_PIN_SET);
```

4.1.5. Matrix_leerTecla

```
static char Matrix_leerTecla (void)
{
    /* Comenzamos inicializando las columnas a 0 por si guardan algún valor */
    for (int fila = 4; fila > 1; fila--) {
        /* Activamos 1 fila en cada bucle */
        HAL_GPIO_WritePin(FILA4_GPIO_Port, FILA4_Pin, (fila == 4));
        HAL_GPIO_WritePin(FILA3_GPIO_Port, FILA3_Pin, (fila == 3));
        HAL_GPIO_WritePin(FILA2_GPIO_Port, FILA2_Pin, (fila == 2));

        /* Leemos columnas */
        if (fila == 4 && HAL_GPIO_ReadPin(COL1_GPIO_Port, COL1_Pin)) {
            Matrix_inicializarFilas();
            return 'A';
        } else if (fila == 4 && HAL_GPIO_ReadPin(COL2_GPIO_Port, COL2_Pin)) {
            Matrix_inicializarFilas();
            return '7';
        } else if (fila == 4 && HAL_GPIO_ReadPin(COL3_GPIO_Port, COL3_Pin)) {
            Matrix_inicializarFilas();
            return '4';
        } else if (fila == 4 && HAL_GPIO_ReadPin(COL4_GPIO_Port, COL4_Pin)) {
            Matrix_inicializarFilas();
            return '1';
        }
```

Esta es la función que se encarga de leer la pulsación de una tecla del teclado matricial. Para ello se usa un método de polling o sondeo en cada una de las filas. Por cada iteración se envía un 1 lógico a una fila (realmente, columna) y luego se comprueba cada columna (fila) para ver si se ha pulsado alguna tecla de la intersección entre dichas fila y columna.

```
/* Volvemos a la fila inicial */
if (fila == 2) {
    fila = 5;
}
```

Este trozo de código comprueba si se ha llegado a la última iteración, volviendo a la primera fila.

4.1.6. Matrix_inicializarFilas

En esta función se envía un 0 lógico a todas las filas (columnas realmente) para que, al llamar a la función anterior, ninguna fila esté activada.

```
static void Matrix_inicializarFilas (void)
{
    HAL_GPIO_WritePin(FILA2_GPIO_Port, FILA2_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(FILA3_GPIO_Port, FILA3_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(FILA4_GPIO_Port, FILA4_Pin, GPIO_PIN_RESET);
}
```

4.1.7. char_to_int

```
static int char_to_int (char tecla) {
    return ((int)tecla - (int)'0');
}
```

Esta es la función encargada de convertir un caracter en un entero, para su posterior uso en el cálculo del precio. Simplemente resta el código ASCII del carácter 0 al código ASCII del carácter correspondiente al parámetro *tecla*.

4.2. NUESTRO BUCLE WHILE

En este último subapartado se explica más detalladamente el bucle while, que es donde radica todo el comportamiento del proyecto:

Antes de comenzar el funcionamiento, se inicializa la placa para comenzar correctamente con su funcionamiento. Así, se inicializan los pines de control para que comiencen deshabilitados, se apagan los cuatro displays y se inicializan las filas (columnas, realmente) para que no contengan ningún valor de comprobación.

```
int main(void)
{
  /* USER CODE BEGIN 1 */

    BCD_inicializarCEx();
    BCD_apagarDisplay();
    Matrix_inicializarFilas();

  /* USER CODE END 1 */
```

```
while (1) {
    char tecla;
    int seleccionado = 0;
    int introducido = 0;
    int parte_entera;
    int parte_decimal;
    double dinero;
    double precio;
    int producto;
    double prueba = (double)1 + (double)2;
    double prueba2 = prueba / 2;
    int fin = 0;
    int signo = 0;
    double devuelta;

    BCD_enviarLetra('H', 4);
    BCD_enviarLetra('O', 3);
    BCD_enviarLetra('L', 2);
    BCD_enviarLetra('A', 1);

    HAL_Delay(2000);
```

Una vez se tiene todo inicializado, comienza el funcionamiento de la Máquina Expendedora. En primer lugar, se definen las variables necesarias durante el proceso, como son la tecla pulsada, si ha sido seleccionado un producto, el dinero introducido y otras que sirven para el cálculo y muestra del dinero en los displays.

Para comenzar el proceso se muestra un mensaje en los displays invocando a la función *BCD_enviarLetra* con los parámetros correspondientes.

A continuación, se muestra el procedimiento para mostrar cada uno de los productos por los displays. Primero se comienza mostrando el primer producto (“ZUMO”) y se espera a que se lea una de las dos teclas de opción, ya sea la *B* para seleccionar o la *A* para mostrar el siguiente producto. Si se pulsa *A* pasa a mostrar el segundo producto (“te”) y, si se vuelve a pulsar *A* se muestra el tercer producto (“COLA”).

Todas estas operaciones se realizan mediante un bucle do-while hasta que se seleccione un producto, mientras tanto se puede ir mostrando uno a uno cada producto. En el momento que se detecte la pulsación de la tecla *B* se actualiza la variable *seleccionado* y la variable *producto*, indicando, de los tres, cuál ha sido seleccionado.

```
do {
    /* Primer producto: ZUMO */
    BCD_enviarLetra('Z', 4);
    BCD_enviarLetra('U', 3);
    BCD_enviarLetra('M', 2);
    BCD_enviarLetra('O', 1);
    tecla = Matrix_leerTecla();

    if (tecla == 'B') {
        seleccionado = 1;
        producto = 1;
        break;
    } else if (tecla == 'A') {
      /* Seguundoo producto: té */
      BCD_enviarLetra('T', 4);
      BCD_enviarLetra('E', 3);
      BCD_enviarLetra('O', 2);
      BCD_enviarLetra('O', 1);
    }
    tecla = Matrix_leerTecla();

    if (tecla == 'B') {
        seleccionado = 1;
        producto = 2;
        break;
    } else if (tecla == 'A') {
      /* Tercer producto: COLA */
      BCD_enviarLetra('C', 4);
      BCD_enviarLetra('O', 3);
      BCD_enviarLetra('L', 2);
      BCD_enviarLetra('A', 1);
    }
    tecla = Matrix_leerTecla();

    if (tecla == 'B') {
        seleccionado = 1;
        producto = 3;
        break;
    }
} while (!seleccionado);
```

```
/* MOSTRAMOS PRECIO SEGÚN PRODUCTO SELECCIONADO */
if (producto == 1) {
    /* 0,95 */
    BCD_enviarDigito(0, 4, 0, 0);
    BCD_enviarDigito(0, 3, 1, 0);
    BCD_enviarDigito(9, 2, 0, 0);
    BCD_enviarDigito(5, 1, 0, 0);

    devuelta = 0.95;
} else if (producto == 2) {
    /* 0,80 */
    BCD_enviarDigito(0, 4, 0, 0);
    BCD_enviarDigito(0, 3, 1, 0);
    BCD_enviarDigito(8, 2, 0, 0);
    BCD_enviarDigito(0, 1, 0, 0);

    devuelta = 0.80;
} else if (producto == 3) {
    /* 1,20 */
    BCD_enviarDigito(0, 4, 0, 0);
    BCD_enviarDigito(1, 3, 1, 0);
    BCD_enviarDigito(2, 2, 0, 0);
    BCD_enviarDigito(0, 1, 0, 0);

    devuelta = 1.20;
}

HAL_Delay(1000);
```

Una vez tenemos un producto seleccionado, se comprueba cuál de los tres es para mostrar su precio por los cuatro displays. Además, se actualiza la variable *devuelta* que será necesaria para el cálculo de la devuelta o el dinero que falte por introducir.

Este y el siguiente trozo de código se encuentran en un bucle do-while que itera hasta que finaliza el correctamente la introducción del dinero y el cálculo de la devuelta.

En primer lugar, se simula la introducción de dinero comprobando que se introducen cuatro dígitos. Internamente lo que se hace es leer la tecla invocando a la función `Matrix_leerTecla` y calcular la parte entera (para los dos primeros dígitos) y la parte decimal (para los dos dígitos restantes) del dinero introducido.

Finalmente, se comprueba si han sido introducido correctamente los cuatro dígitos y se actualiza la variable *introducido*, que indica el fin de este proceso.

```
/* ESPERAMOS A INTRODUCIR EL DINERO */
do {
    while (!introducido) {
        int digitos = 0;
        tecla = Matrix_leerTecla();
        parte_entera = char_to_int(tecla) * 10;

        /* Comprobamos que se ha pulsado un número */
        if (tecla >= '0' || tecla <= '9') {
            ++digitos;
        }
        HAL_Delay(500);

        tecla = Matrix_leerTecla();
        parte_entera += char_to_int(tecla);

        /* Comprobamos que se ha pulsado un número */
        if (tecla >= '0' || tecla <= '9') {
            ++digitos;
        }
        HAL_Delay(500);

        tecla = Matrix_leerTecla();
        parte_decimal = char_to_int(tecla) * 10;

        /* Comprobamos que se ha pulsado un número */
        if (tecla >= '0' || tecla <= '9') {
            ++digitos;
        }
        HAL_Delay(500);

        tecla = Matrix_leerTecla();
        parte_decimal += char_to_int(tecla);

        /* Comprobamos que se ha pulsado un número */
        if (tecla >= '0' || tecla <= '9') {
            ++digitos;
        }
        HAL_Delay(500);

        if (digitos == 4) {
            introducido = 1;
        }
    }
```

```
    /* Mostramos dinero introducido */
    BCD_enviarDigito(parte_entera / 10, 4, 0, 0);
    BCD_enviarDigito(parte_entera % 10, 3, 1, 0);
    BCD_enviarDigito(parte_decimal / 10, 2, 0, 0);
    BCD_enviarDigito(parte_decimal % 10, 1, 0, 0);

    HAL_Delay(2000);

    /* Obtenemos el dinero introducido para calcular la devuelta */
    dinero = (double)parte_entera * 100 + (double)parte_decimal;
    dinero = dinero / 100;

    /* MOSTRAMOS DINERO QUE FALTA O DEVUELTA */
    if (producto == 1) {    // ZUMO
        devuelta = devuelta - dinero;
    } else if (producto == 2) {    // TE
        devuelta = devuelta - dinero;
    } else if (producto == 3) {    // COLA
        devuelta = devuelta - dinero;
    }

    if (devuelta <= 0) {
        devuelta *= (-1);    // Quitamos signo
        signo = 1;
        fin = 1;
    } else {
        introducido = 0;
    }

    parte_entera = devuelta;
    parte_decimal = (devuelta * 100);
    parte_decimal = parte_decimal % 100;

    BCD_enviarDigito(parte_entera / 10, 4, 0, signo);
    BCD_enviarDigito(parte_entera % 10, 3, 1, signo);
    BCD_enviarDigito(parte_decimal / 10, 2, 0, signo);
    BCD_enviarDigito(parte_decimal % 10, 1, 0, signo);

} while (!fin);

HAL_Delay(5000);
```

Una vez hemos introducido una cantidad de dinero se muestra dicha cantidad por los cuatro displays durante 2 segundos y, a continuación, se calcula el valor de devuelta. Si este valor es superior a 0 indica que el precio introducido ha sido menor al precio del producto, y por tanto debemos seguir introduciendo monedas. En cambio, si la devuelta es 0 o menor, dicho valor se mostrará por los displays (junto al led de signo activado) para indicar el dinero que debe devolver la máquina. Es en este momento cuando se actualiza la variable *fin*, indicando el fin del pago del producto.

Ya terminado el proceso, lo que se hace es apagar el led de sigo para la siguiente petición de productos, siempre en un ciclo de reloj. Justo después, vuelve a mostrarse el mensaje de bienvenida.

```
    HAL_GPIO_WritePin(SIGNO_GPIO_Port, SIGNO_Pin, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_RESET);
    HAL_Delay(10);
    HAL_GPIO_WritePin(DISPCLK_GPIO_Port, DISPCLK_Pin, GPIO_PIN_SET);
    HAL_Delay(10);

}
```